



UNIVERSITÀ DEGLI STUDI DI PERUGIA
Dipartimento di Ingegneria

CORSO DI LAUREA TRIENNALE IN
INGEGNERIA INFORMATICA ED ELETTRONICA

Tesi di Laurea

**Sviluppo di un'applicazione Web responsive
per la visualizzazione di previsioni meteorologiche**

Laureando:
Michele Mariotti

Relatore:
Prof. Luca Grilli

Anno Accademico 2022/2023

Indice

1	Introduzione	5
2	Concetti e tecnologie preliminari	8
2.1	Responsive design	8
2.2	HTML	9
2.3	CSS	10
2.4	JavaScript	12
2.4.1	Moduli ES6	12
2.5	API REST	12
2.6	JSON	13
3	Il problema affrontato	15
3.1	I dati meteorologici	16
3.1.1	I dati meteorologici giornalieri	16
3.1.2	I dati meteorologici orari	18
3.2	Specifica dei requisiti	19
3.2.1	Elenco dei requisiti funzionali	19
3.2.2	Elenco dei requisiti non funzionali	20
4	L'applicazione UniPG Weather Forecast	21
4.1	I layout dell'interfaccia utente dell'applicazione	21

INDICE	2
4.1.1 Layout desktop	22
4.1.2 Layout mobile	25
4.2 Architettura del software	28
4.3 Struttura e aspetto delle pagine	30
4.3.1 Struttura delle pagine	30
4.3.2 Aspetto delle pagine	31
4.4 Interattività e dinamicità delle pagine	32
4.4.1 Model	33
4.4.2 View	35
4.4.3 Controller	36
4.4.4 File esterni alle componenti	38
5 Risultato finale ed analisi di alcuni casi d'uso	39
6 Test	47
6.1 Validazione HTML	47
6.2 Test di efficienza	47
7 Conclusioni e sviluppi futuri	51
7.1 Sviluppi futuri	52
Bibliografia	54

Elenco delle figure

2.1	Esempio di interfaccia responsive su dispositivo mobile [3]	9
2.2	Esempio di elemento HTML	9
2.3	Esempio di regola CSS	11
2.4	Esempio di media queries [6]	11
2.5	Esempio di JSON	14
3.1	Tabella fornitori nazionali dati meteo [13]	16
3.2	Conversione gradi in punti cardinali [14]	17
4.1	Wireframe desktop pagina previsioni giornaliere	23
4.2	Wireframe desktop pagina previsioni orarie	24
4.3	Wireframe mobile pagina previsioni giornaliere	26
4.4	Wireframe mobile pagina previsioni orarie	27
4.5	Struttura del software	29
4.6	Codice compattato del corpo della pagina <i>index.html</i>	30
4.7	Interazione tra le componenti MVC	33
5.1	Pagina iniziale previsioni giornaliere layout desktop	40
5.2	Selezione giornata dati secondari	40
5.3	Pagina iniziale previsioni giornaliere layout mobile	41
5.4	Impostazioni ed auto-completamento ricerca su dispositivi mobili	42

5.5	Pagina previsioni orarie layout desktop	43
5.6	Pagina previsioni orarie layout mobile	44
5.7	Icone condizioni meteo prima e dopo l'alba	45
5.8	Footer della pagina	46
6.1	“Bollino verde” validazione HTML	47
6.2	Tabelle comparative risultati test	50

Capitolo 1

Introduzione

Le previsioni del meteo sono un servizio utilizzato da molti utenti quotidianamente, al punto da diventare uno strumento cruciale per pianificare le attività giornaliere.

Tuttavia, nonostante l'importanza di questo servizio, molti utenti lamentano problemi legati alla pubblicità invasiva e all'usabilità non ottimale dei principali portali che offrono previsioni meteorologiche.

Le interfacce utente di alcune di queste piattaforme risultano essere poco user-friendly, compromettendo la fruizione immediata delle informazioni meteorologiche. Invece di agevolare un accesso rapido ai dati maggiormente consultati, incoraggiano un'analisi più dettagliata della situazione meteo, a cui spesso l'utente non è interessato.

Un esempio concreto è l'attuale GUI del portale dell'aeronautica, che, seppur sofisticata, complica l'accesso alle informazioni meteorologiche fondamentali. La sua complessità emerge soprattutto nell'offrire strumenti avanzati per analisi dettagliate, utilizzando visualizzazioni che sovrappongono su mappe geografiche varie icone indicative delle condizioni meteo.

Questa interfaccia è stata criticata da molti utenti abituati alla vecchia

versione, che risultava essere minimale ma ottima per reperire le previsioni in modo rapido [1]. Gli utenti lamentano la perdita dell'immediatezza che contraddistingueva la prima versione. Nella nuova versione risulta necessario effettuare diverse interazioni anche per operazioni elementari, come ottenere le previsioni per la propria città.

Un altro aspetto che compromette l'esperienza dell'utente è la presenza di pubblicità invasiva. Questo elemento non solo rappresenta un disturbo costante, distogliendo l'attenzione dell'utente dalle informazioni cercate, ma rende anche meno efficiente il servizio, andando a rallentare il caricamento delle pagine.

Gli utenti sono interessati a consultare i dati di interesse in modo chiaro ed immediato, attraverso un'interfaccia che permetta loro di avere un quadro sintetico delle previsioni.

È questo l'obiettivo che si pone la presente tesi: realizzare un'applicazione Web gratuita, usufruibile sia su dispositivi mobili che desktop, che soddisfi le richieste degli utenti attraverso un'interfaccia responsive, user-friendly e priva di pubblicità.

Il documento di tesi è organizzato nei seguenti capitoli:

Capitolo 2 - Concetti e tecnologie preliminari: In questo capitolo vengono descritti i concetti e le tecnologie alla base della realizzazione dell'applicazione, riportando le informazioni strettamente necessarie per la comprensione del resto del documento.

Capitolo 3 - Il problema affrontato: Nel capitolo si analizza il problema affrontato. Inoltre vengono definiti i requisiti del software.

Capitolo 4 - L'applicazione UniPG Weather Forecast: In questo capitolo viene illustrato l'iter progettuale e l'architettura del software, de-

scrivendo il ruolo delle varie componenti dell'applicazione realizzata.

Capitolo 5 - Risultato finale ed analisi di alcuni casi d'uso: Nel capitolo sono riportate e commentate alcune schermate del risultato finale ottenuto a seguito dell'implementazione.

Capitolo 6 - Test: Nel capitolo sono presentati i test condotti al fine di valutare alcuni aspetti dell'applicazione.

Capitolo 7 - Conclusioni e sviluppi futuri: In questo capitolo vengono riassunti i passi principali della realizzazione della tesi e gli obiettivi raggiunti. Inoltre vengono ipotizzati possibili sviluppi futuri.

Capitolo 2

Concetti e tecnologie preliminari

Sono introdotti di seguito i concetti e le tecnologie necessarie per la comprensione del resto del documento.

2.1 Responsive design

Il responsive design [2] è un approccio alla progettazione e allo sviluppo di applicazioni e siti Web che mira a garantire all'utente un'esperienza ottimale su dispositivi con schermi di diverse dimensioni e risoluzioni.

Questo termine viene utilizzato per riferirsi ad un insieme di tecniche e best practices utilizzate per realizzare un'interfaccia grafica in grado di adattarsi a qualsiasi dispositivo mantenendo una qualità adeguata. Più precisamente il contenuto deve adattarsi al **viewport**, ovvero all'area visibile della pagina web nel browser del dispositivo dell'utente. Questa grandezza varia in base alle dimensioni della finestra del browser e può cambiare quando l'utente ridimensiona la finestra o ruota il dispositivo.

Due esempi di tecniche utilizzate sono l'impiego di immagini vettoriali, che possono essere ingrandite o ridotte senza perdita di qualità, e le **media queries** (vedi paragrafo 2.3).



Figura 2.1: Esempio di interfaccia responsive su dispositivo mobile [3]

2.2 HTML

HTML (HyperText Markup Language) [4] è un linguaggio di markup utilizzato per definire la struttura delle pagine Web.

Il componente base della sintassi di questo linguaggio è l'**elemento**, delimitato da marcature dette **tag**.

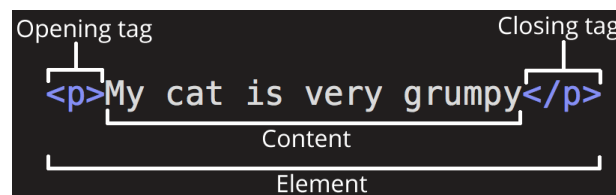


Figura 2.2: Esempio di elemento HTML

Esistono due tipologie di tag, quelli **semantici**, che forniscono informazioni sul significato e sulla funzione dell'elemento, migliorando l'accessibilità della pagina, e quelli **generici** (o non semantici), che non specificano nulla sulla semantica del contenuto al loro interno. Un esempio di tag semantico è `<header>`, da cui possiamo dedurre che il contenuto rappresenta l'intestazione della pagina, mentre un esempio di tag non semantico è `<div>`, ovvero un generico contenitore.

Gli elementi di base, indispensabili all'interno di un file HTML, sono:

- `<html>`: è l'elemento radice, contiene tutti gli altri elementi.
- `<head>`: contiene le informazioni sul documento, come il titolo della pagina, i metadati ed i collegamenti a file CSS e JavaScript.
- `<body>`: include il contenuto, ovvero i vari elementi che andranno a comporre la struttura della pagina.

2.3 CSS

CSS (Cascading Style Sheets) [5] è un linguaggio utilizzato per definire l'aspetto delle pagine. Descrive come gli elementi HTML devono essere visualizzati, determinando aspetti visivi come layout, colori, tipografia, posizionamento e molti altri. Per farlo utilizza delle **regole**, ovvero seleziona l'elemento e ne definisce alcune proprietà attraverso una dichiarazione.

CSS fornisce una funzionalità fondamentale per adattare lo stile di una pagina in base al viewport su cui viene visualizzata: le **media queries**. Questo strumento, utilizzato per rendere l'applicazione responsive, consiste in una regola che viene richiamata quando il viewport ha determinate dimensioni definite nel selettore. Al suo interno vengono definite altre regole che cambiano

l'aspetto della pagina con lo scopo di adattare il contenuto alle dimensioni del viewport.

Supponendo idealmente di avere un viewport di 0 x 0 pixel e di ridimensionarlo andando ad aumentarne progressivamente la larghezza e l'altezza, tutti i punti in cui avviene un cambiamento, determinato da una media query, vengono chiamati **breakpoints**. Nell'esempio della figura 2.4 i breakpoints, riguardanti la larghezza del viewport, sono 768 px e 1024 px.

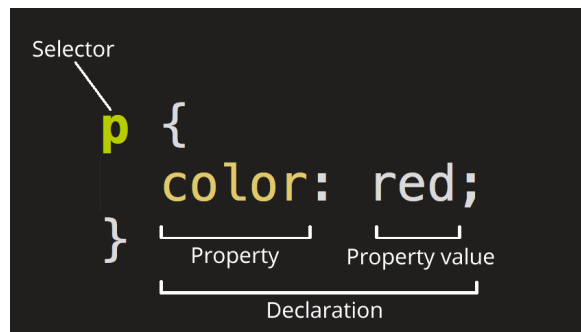


Figura 2.3: Esempio di regola CSS

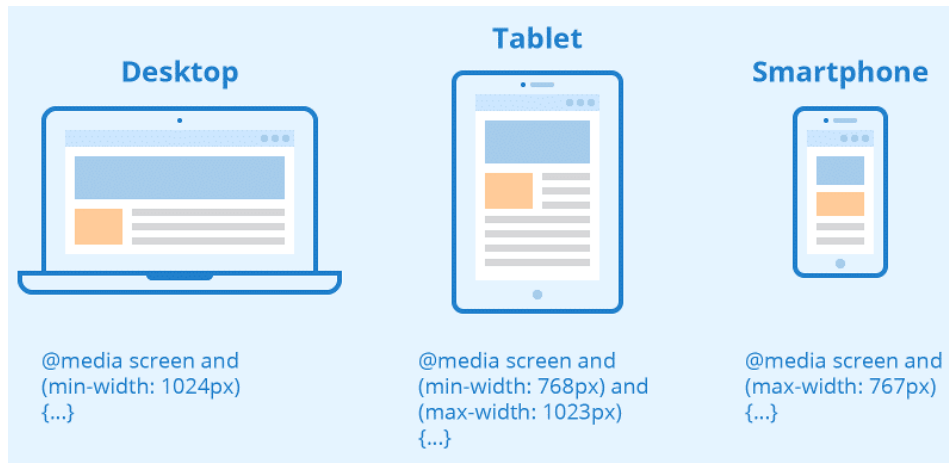


Figura 2.4: Esempio di media queries [6]

2.4 JavaScript

JavaScript [7] è un linguaggio di programmazione **multi-paradigma**, ovvero supporta diversi approcci alla programmazione, inclusi paradigmi come la programmazione a eventi, la programmazione orientata agli oggetti, la programmazione funzionale e la programmazione imperativa.

Questo linguaggio viene interpretato durante l'esecuzione ed è caratterizzato da una tipizzazione debole.

È ampiamente utilizzato nello sviluppo Web lato client per aggiungere interattività e funzionalità dinamiche alle pagine.

2.4.1 Moduli ES6

A partire da ECMAScript 2015 (ES6), JavaScript supporta nativamente i moduli, che consentono di organizzare e strutturare il codice in modo modulare [8].

Per creare i moduli è sufficiente dividere il codice in più file ed esportare funzioni, variabili e oggetti da un file per poi importarli in altri che ne usufruiscono, attraverso le dichiarazioni *export* e *import*.

Esiste una corrispondenza 1 a 1 tra i file ed i moduli, infatti ogni file corrisponde a un modulo e un file può contenere un solo modulo [9].

Questo permette di suddividere il codice in moduli separati, migliorandone la manutenibilità e la leggibilità.

2.5 API REST

Una API (Application Programming Interface) è una procedura che consente a un'applicazione (client) di accedere ad una **risorsa** all'interno di un'altra

applicazione (server).

Una API REST [10] è una API che segue i principi dell'architettura REST (Representational State Transfer).

Nell'architettura REST ogni singola risorsa è identificata univocamente dal proprio URL ed è manipolabile attraverso quattro operazioni: creazione, lettura, aggiornamento e cancellazione. Queste operazioni vengono mappate rispettivamente nei seguenti quattro metodi offerti da HTTP(S): POST, GET, PUT e DELETE.

Una risorsa può avere diverse rappresentazioni, tipicamente viene rappresentata nel formato XML o JSON, quest'ultimo viene descritto nel paragrafo seguente.

2.6 JSON

JSON (JavaScript Object Notation) [11] è un formato per lo **scambio di dati** semplice e leggero. È nativamente supportato da JavaScript in quanto il suo formato è ispirato alla sintassi degli oggetti in JavaScript, di conseguenza è facilmente convertibile in un suo oggetto.

I dati sono rappresentati mediante coppie chiave-valore, organizzati in oggetti (delimitati da parentesi graffe) o elenchi (delimitati da parentesi quadre).

Nell'esempio in figura 2.5 il JSON contiene i campi *latitude*, *longitude* e *timezone* con i rispettivi valori. Inoltre è presente un oggetto *daily* al cui interno si trovano i tre elenchi *time*, *temperature_2m_max* e *temperature_2m_min* contenenti dei dati.

```
1  {
2    "latitude": 43.11,
3    "longitude": 12.389999,
4    "timezone": "Europe/Rome",
5    "daily": {
6      "time": [
7        "2023-11-06",
8        "2023-11-07",
9        "2023-11-08"
10     ],
11     "temperature_2m_max": [
12       15.7,
13       12.9,
14       12.9
15     ],
16     "temperature_2m_min": [
17       10.5,
18       8.6,
19       6.0
20     ]
21   }
22 }
```

Figura 2.5: Esempio di JSON

Capitolo 3

Il problema affrontato

Il problema affrontato è la realizzazione di un'applicazione Web che permetta di visualizzare dati meteorologici attraverso un'interfaccia responsive e user-friendly.

Tra i principali vantaggi del realizzare un'applicazione Web, invece di un'applicazione nativa per dispositivi mobili o desktop, abbiamo la portabilità della stessa, che sarà accessibile tramite browser in qualsiasi dispositivo con una connessione ad Internet. Questa ampia portabilità rende di estrema importanza la realizzazione di un'interfaccia responsive, che permetta di visualizzare adeguatamente i dati sia su dispositivi mobili che desktop.

Inoltre, un ulteriore vantaggio è l'utilizzo praticamente nullo della memoria secondaria del sistema, dato che non necessita di alcuna installazione.

Per poter scegliere come visualizzare i dati meteorologici è necessario prima definire quali dati saranno disponibili e specificare i requisiti del software.

3.1 I dati meteorologici

I dati meteorologici sono forniti dalle API REST di *Open-Meteo*, che fornisce tali dati in maniera gratuita per uso non commerciale garantendo fino a 10'000 chiamate giornaliere. Questo permette all'applicazione di fornire un servizio gratuito, sostenibile senza l'utilizzo di pubblicità.

Le API di *Open-Meteo* utilizzano modelli meteorologici di più fornitori nazionali. Per ciascuna località nel mondo, i migliori modelli vengono combinati per fornire la migliore previsione possibile [12]. Vengono fornite previsioni giornaliere ed orarie fino a 16 giorni.

Weather Model	National Weather Provider	Origin Country	Resolution	Forecast Length	Update frequency
ICON	Deutscher Wetterdienst (DWD)	Germany	2 - 11 km	7.5 days	Every 3 hours
GFS	NOAA	United States	3 - 25 km	16 days	Every hour
Arpege & Arome	MeteoFrance	France	1 - 40 km	4 days	Every 6 hours
IFS	ECMWF	European Union	44 km	7 days	Every 6 hours
JMA	JMA	Japan	5 - 55 km	11 days	Every 3 hours
MET Nordic	MET Norway	Norway	1 km	2.5 days	Every hour
GEM	Canadian Weather Service	Canada	2.5 km	10 days	Every 6 hours

Figura 3.1: Tabella fornitori nazionali dati meteo [13]

Prima di iniziare la realizzazione del progetto è stata necessaria una fase di selezione e classificazione dei dati ottenibili, che sono stati divisi in dati giornalieri ed orari, a loro volta divisi in dati primari e secondari (in base alla frequenza con cui gli utenti solitamente consultano questi dati) come illustrato in seguito.

3.1.1 I dati meteorologici giornalieri

Dati primari

- condizioni meteo (codice da convertire in immagine e descrizione)

- temperatura massima (in gradi Celsius o Fahrenheit)
- temperatura minima (in gradi Celsius o Fahrenheit)
- somma precipitazioni (in mm o inch)
- massima probabilità di precipitazioni
- velocità massima vento (in km/h, m/s, mph o kn)
- direzione dominante vento (in gradi, da convertire in punto cardinale)

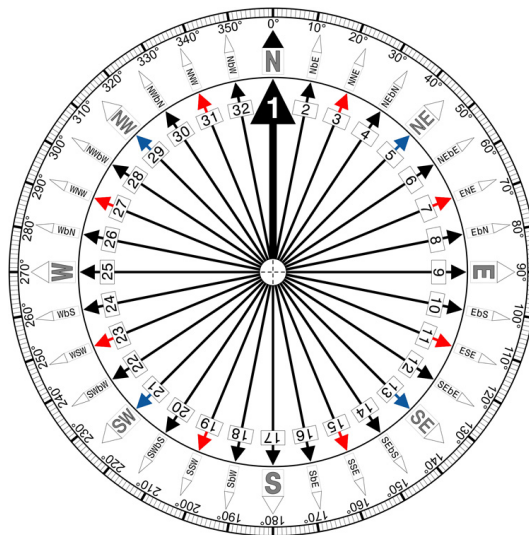


Figura 3.2: Conversione gradi in punti cardinali [14]

Dati secondari

- temperatura massima apparente (in gradi Celsius o Fahrenheit)
- temperatura minima apparente (in gradi Celsius o Fahrenheit)
- orario alba
- orario tramonto

- velocità massima raffiche di vento (in km/h, m/s, mph o kn)
- indice UV massimo

3.1.2 I dati meteorologici orari

Dati primari

- condizioni meteo (codice da convertire in immagine e descrizione)
- temperatura (in gradi Celsius o Fahrenheit)
- temperatura apparente (in gradi Celsius o Fahrenheit)
- precipitazioni (in mm o inch)
- probabilità di precipitazioni
- velocità vento (in km/h, m/s, mph o kn)
- direzione vento (in gradi, da convertire in punto cardinale)

Dati secondari

- umidità relativa
- pressione (in hPa)
- velocità massima raffiche di vento (in km/h, m/s, mph o kn)

3.2 Specifica dei requisiti

3.2.1 Elenco dei requisiti funzionali

Si riportano in seguito i requisiti funzionali, ovvero un elenco delle funzionalità offerte dall'applicazione.

L'applicazione deve:

1. mostrare i dati giornalieri delle previsioni del meteo.
2. mostrare i dati orari delle previsioni del meteo, evitando di mostrare quelli relativi alle ore passate.
3. mostrare la condizione meteo e la temperatura attuale.
4. permettere la scelta della città di cui si vogliono visualizzare le previsioni meteorologiche.
5. suggerire l'auto-completamento della città ricercata.
6. fornire la possibilità di navigare tra le pagine.
7. permettere di impostare la lingua del contenuto testuale al suo interno, compreso il nome della città di cui si vogliono visualizzare i dati.
8. permettere di impostare le unità di misura della temperatura, della velocità del vento e dell'altezza pluviometrica.
9. memorizzare le impostazioni scelte.
10. memorizzare l'ultima città ricercata.

3.2.2 Elenco dei requisiti non funzionali

Vengono riportati in seguito i requisiti non funzionali, ovvero dei vincoli sulle funzionalità offerte dall'applicazione.

L'interfaccia dell'applicazione deve:

1. essere responsive, garantendo un'esperienza ottimale sia su dispositivi mobile (a partire da una larghezza minima del viewport di 320 px) che desktop.
2. essere user-friendly, con un layout semplice ed intuitivo.
3. enfatizzare i dati più importanti.

L'applicazione deve:

4. garantire una buona efficienza nel caricamento delle pagine.
5. mostrare un messaggio di errore nel caso in cui ci siano dei problemi nel caricamento dei dati.
6. utilizzare file HTML validati attraverso un apposito servizio.

Capitolo 4

L'applicazione

UniPG Weather Forecast

In questo capitolo illustrerò gli aspetti riguardanti la progettazione e la realizzazione dell'applicazione. Si descriveranno i wireframe dell'interfaccia utente, l'architettura del software ed infine i vari aspetti realizzativi.

4.1 I layout dell'interfaccia utente dell'applicazione

Il primo passo nella realizzazione dell'applicazione è stata la scelta del layout dell'interfaccia utente.

L'obiettivo di questa fase di progettazione è stato quello di rispondere alla domanda “come visualizzare i dati?”.

A tale scopo ho realizzato dei **wireframe**, ovvero dei modelli preliminari dell'interfaccia utilizzati per definirne la struttura di base senza dettagli grafici o elementi di design. Vengono utilizzati nel processo di progettazione dell'in-

terfaccia per pianificare la disposizione degli elementi e l'organizzazione delle informazioni prima di passare alla fase di sviluppo vero e proprio.

Al fine di rispettare gli obiettivi prefissati (vedi paragrafo 3.2), ho scelto di realizzare due layout distinti, uno per dispositivi mobili ed uno per dispositivi desktop.

Sono state previste due pagine, una principale dedicata alle previsioni giornaliere, mentre l'altra dedicata alle previsioni orarie.

Inoltre ho scelto di organizzare i dati in un formato tabellare, in modo tale da garantire un layout semplice e pulito, che permetta all'utente di orientarsi in modo immediato all'interno della pagina, dando la priorità ai dati più spesso presi in considerazione, attraverso una struttura rigorosa e ben definita.

In seguito vengono illustrati nel dettaglio i wireframe realizzati dei due layout.

4.1.1 Layout desktop

Il layout illustrato in questo paragrafo viene visualizzato nei dispositivi desktop. Come detto precedentemente, sono previste due pagine, una per le previsioni giornaliere ed una per quelle orarie, per questo sono stati realizzati due wireframe.

In entrambe le pagine l'header ha la seguente struttura: sono presenti a sinistra il logo del sito, al centro il nome della città con il relativo meteo attuale e a destra una barra di ricerca, che permette di inserire la città di cui si vogliono vedere le previsioni meteo, affiancata da un pulsante che consente di modificare le impostazioni.

Nella pagina dedicata alle previsioni meteo giornaliere, il contenuto principale è una tabella che riporta i dati primari (vedi sotto-paragrafo 3.1.1) organizzati in forma tabellare.

Nel lato destro della pagina, in un'altra tabella di dimensioni ridotte, vengono mostrati i dati secondari. In modo predefinito si mostrano i dati della giornata odierna, per visionare i dati di una determinata giornata sarà sufficiente cliccare sull'icona contenente il simbolo +, posizionata alla fine della relativa riga. Questo contenuto laterale sarà presente anche quando si scorre la pagina in basso, garantendo la possibilità di visionare i dati secondari della giornata posizionata ad esempio nell'ultima riga della tabella, senza dover ritornare ad inizio pagina.

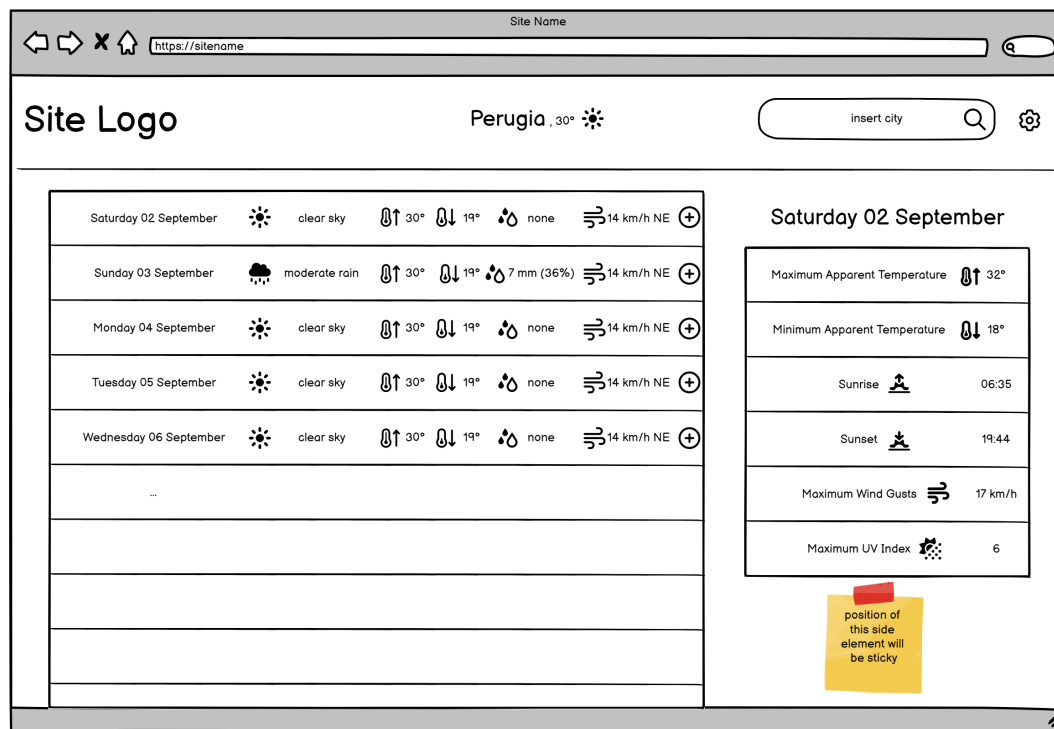


Figura 4.1: Wireframe desktop pagina previsioni giornaliere

Per accedere alla pagina delle previsioni orarie sarà sufficiente cliccare in un qualsiasi punto all'interno della riga relativa alla giornata a cui si è interessati.

La struttura è analoga a quanto descritto precedentemente nel caso delle previsioni giornaliere, con il contenuto principale che consiste in una tabella

in cui vengono riportati i dati primari (vedi sotto-paragrafo 3.1.2), ed un'altra tabella più piccola contenente i dati secondari posizionata sulla destra. In questo caso cliccando le icone con il simbolo + si seleziona l'ora di cui si vogliono vedere i dati secondari nella tabella a lato.

È previsto un widget che permetta di cambiare il giorno di cui si vogliono visualizzare le previsioni orarie senza dover tornare alla pagina iniziale delle previsioni giornaliere.

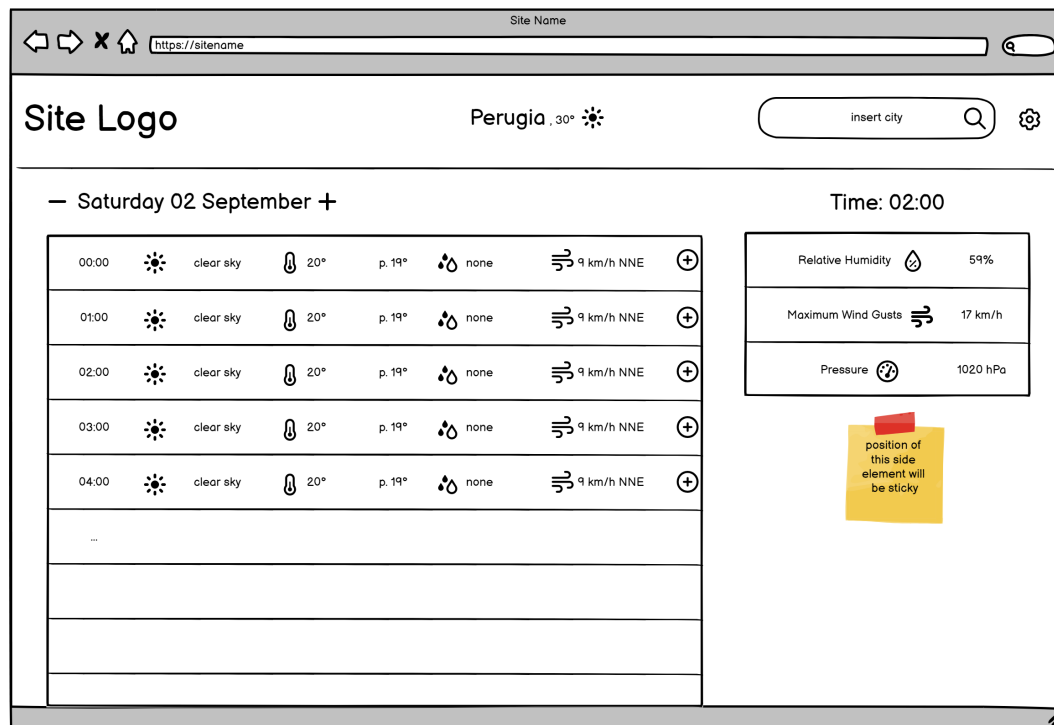


Figura 4.2: Wireframe desktop pagina previsioni orarie

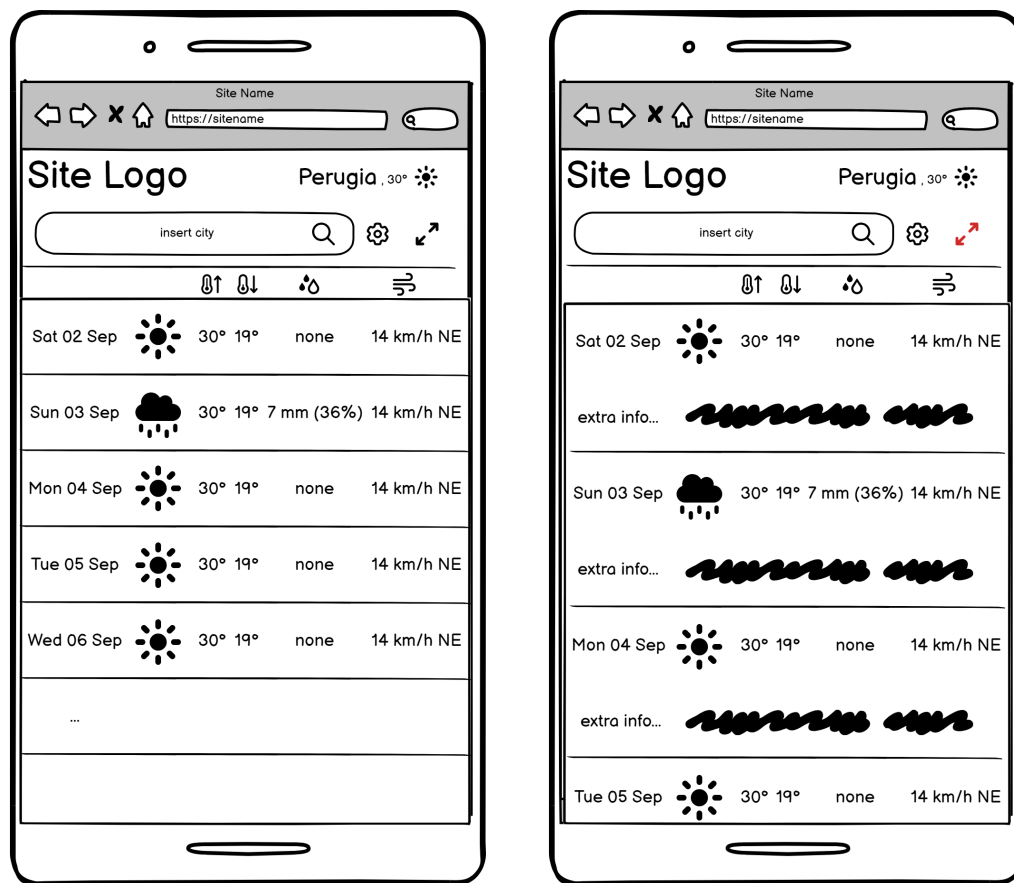
4.1.2 Layout mobile

Il layout illustrato in questo paragrafo viene visualizzato nei dispositivi mobili.

Tutte le considerazioni fatte per la versione desktop valgono anche in questo caso, il principale cambiamento riguarda il compattamento degli elementi e dei dati, al fine di rientrare all'interno della larghezza del viewport mantenendo l'interfaccia chiara.

Le tabelle laterali contenenti i dati secondari sono sostituite da righe aggiuntive nella tabella principale, che potranno essere rese visibili o nascoste attraverso un pulsante posizionato nell'header.

L'header è suddiviso verticalmente in due sezioni, con il logo e la città con il relativo meteo attuale nella parte superiore, mentre nella parte inferiore abbiamo la barra di ricerca, il pulsante delle impostazioni ed il pulsante che permette di espandere/restringere il contenuto della tabella.



(a) Versione ristretta

(b) Versione espansa

Figura 4.3: Wireframe mobile pagina previsioni giornaliere

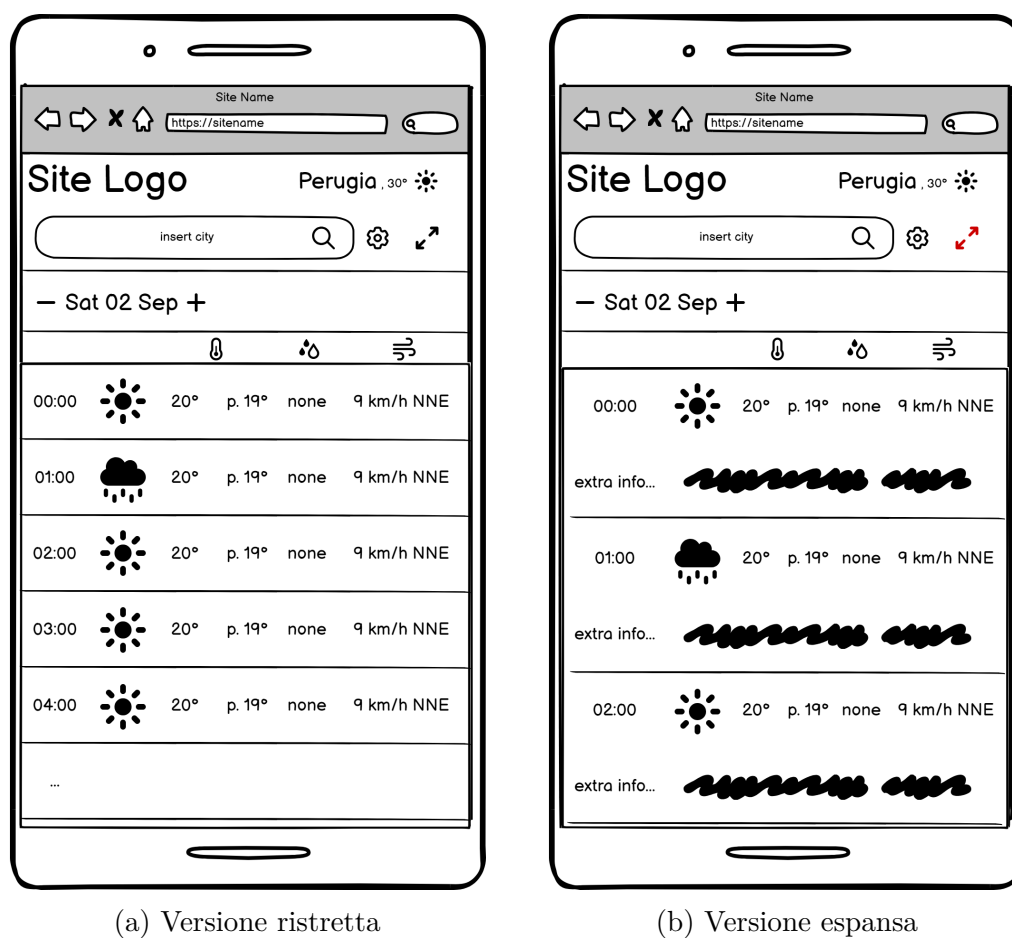


Figura 4.4: Wireframe mobile pagina previsioni orarie

4.2 Architettura del software

Il software è stato progettato utilizzando un'architettura ben strutturata che segue il principio di separazione degli interessi, favorendo la manutenibilità del codice e l'evolubilità dell'applicazione.

La cartella principale contiene le seguenti sottocartelle:

- **images:** questa cartella è dedicata alle risorse grafiche dell'applicazione, come immagini ed icone. Per realizzarle è stato adoperato *Adobe Illustrator*, utilizzando alcuni elementi di base non protetti da copyright o richiesta di attribuzione. Una volta create le immagini sono state esportate nel formato svg per garantire la loro qualità a qualsiasi dimensione.
- **pages:** contiene i file HTML che costituiscono le pagine dell'applicazione (con l'eccezione del file *index.html*). Al momento contiene solamente la pagina *hourly-weather.html*, ma è stato comunque scelto di mantenere questa cartella in modo tale che eventuali pagine aggiunte in futuro possano essere inserite al suo interno.
- **styles:** all'interno di questa cartella sono presenti i file CSS che definiscono lo stile e l'aspetto dell'applicazione. La separazione tra HTML e CSS è fondamentale per una gestione più efficiente dei design e dei layout.
- **scripts:** questa cartella contiene i file JavaScript, responsabili dell'interattività e della dinamicità della pagina. È ulteriormente suddivisa in tre sottocartelle: *model*, *view*, e *controller*. Questa suddivisione adotta il design pattern MVC (Model-View-Controller), ampiamente utilizzato per organizzare il codice in modo ordinato ed efficiente:

- **model**: contiene i file JavaScript che si occupano di ottenere i dati utilizzati nell'applicazione.
- **view**: ospita i file JavaScript che si occupano di aggiornare le pagine HTML in base ai dati provenienti dal *model* e di fornire al *controller* alcune funzioni per gestire aspetti dell'interfaccia.
- **controller**: i file JavaScript al suo interno si occupano di gestire le interazioni dell'utente e la navigazione tra le pagine dell'applicazione.

Inoltre, all'interno della cartella principale, è presente il file **index.html**, che funge da punto di ingresso principale dell'applicazione Web.

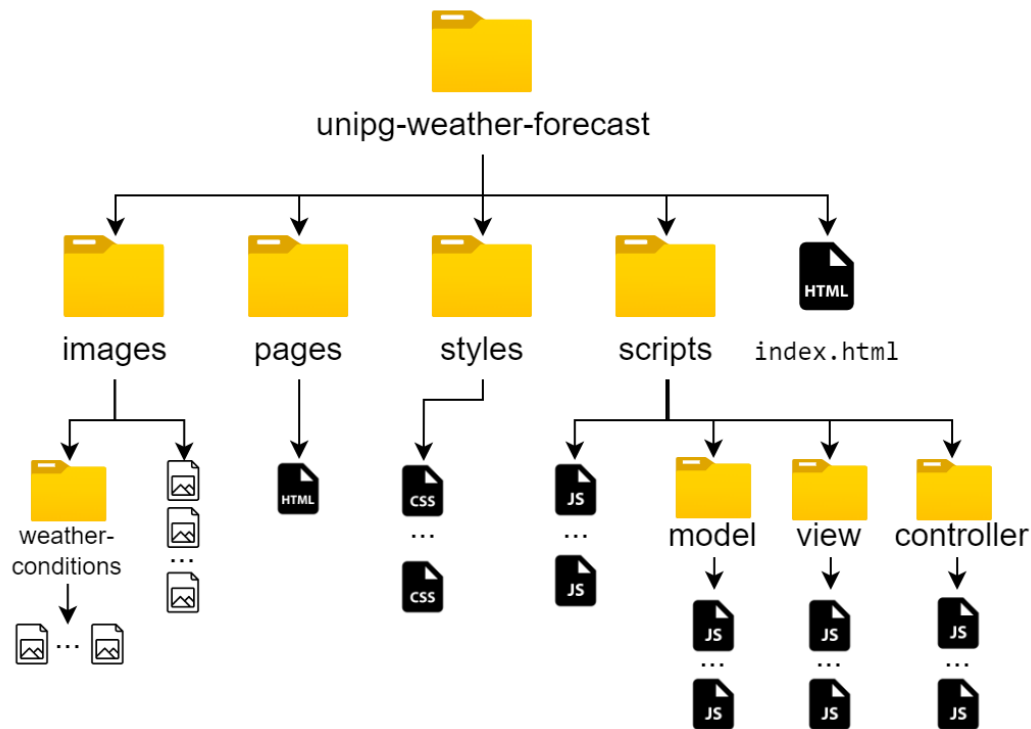


Figura 4.5: Struttura del software

4.3 Struttura e aspetto delle pagine

4.3.1 Struttura delle pagine

Il primo passo realizzativo è stato l'implementazione della struttura delle pagine, scelta precedentemente nei wireframe, tramite il linguaggio di markup HTML. Come previsto sono state realizzate due pagine, *index.html*, localizzata nella root folder, e *hourly-weather.html*, posizionata all'interno della cartella *pages*. La prima è la pagina iniziale dell'applicazione, in cui vengono visualizzate le previsioni giornaliere, mentre *hourly-weather.html* è la pagina in cui vengono mostrate le previsioni orarie.

Nell'head delle pagine sono stati collegati i file CSS e JavaScript necessari per definire il layout ed il comportamento delle pagine, mentre nel body si è tradotto in codice l'idea di layout definita nei wireframe.

```
30  <body>
31    <header> ...
85    </header>
86    <main id="index-main">
87      <div id="main-container">
88        <table id="main-table"> ...
1225      </table>
1226    </div>
1227    <aside>
1228      <h2 id="extra-info-date">loading date...</h2>
1229      <table id="aside-table"> ...
1296    </table>
1297    </aside>
1298  </main>
1299  <footer id="footer"> ...
1308  </footer>
1309 </body>
```

Figura 4.6: Codice compattato del corpo della pagina *index.html*

4.3.2 Aspetto delle pagine

Il secondo passo è stato quello di definire l'aspetto e la formattazione delle pagine tramite il linguaggio CSS.

A tale scopo è stato utilizzato l'approccio *Mobile First* [15], ovvero è stata data inizialmente la priorità allo sviluppo del layout mobile, a discapito del layout desktop. Questa metodologia mira a garantire che gli utenti su dispositivi mobili abbiano un'esperienza ottimale e che l'applicazione sia più veloce e leggera per tali dispositivi.

Le regole CSS utilizzate per ottenere l'aspetto desiderato sono state divise in 7 file, in modo tale da rendere più facili le modifiche, situati nella cartella *styles*:

- **header-style.css**: si occupa di definire l'aspetto e la formattazione dell'header di entrambe le pagine.
- **index-main-style.css**: contiene le regole utilizzate per definire l'aspetto del `<main>` della pagina *index.html*.
- **hourly-weather-main-style.css**: gestisce l'aspetto del `<main>` della pagina *hourly-weather.html*
- **media-queries.css**: contiene le modifiche da effettuare all'aumentare del viewport. Grazie alle media queries contenute in questo file avviene il passaggio al layout desktop quando il viewport è di larghezza superiore o uguale a 1025 px e altezza superiore o uguale a 500 px. Quest'ultima condizione è stata posta per garantire che questo layout sia adottato solamente se tutti i dati nella tabella posizionata a lato sono visibili. Piccoli cambiamenti per migliorare l'esperienza su dispositivi come i tablet sono previsti nei contenuti visualizzati su viewport di larghezza compresa tra i 768 px ed i 1279 px (estremi inclusi).

- **footer-style.css**: si occupa di definire l'aspetto e la formattazione del footer.
- **basic-elements-style.css**: contiene delle regole base.
- **colors.css**: contiene delle variabili utilizzate per definire i colori degli elementi. Il suo scopo è quello di permettere facilmente di variare i colori cambiando i valori assegnati alle variabili, senza dover andare a modificare le singole regole.

4.4 Interattività e dinamicità delle pagine

Il terzo ed ultimo passo è stato l'implementazione dell'interattività e delle funzionalità dinamiche delle pagine.

A tale scopo sono stati realizzati dei file JavaScript che, come anticipato precedentemente, sono stati divisi in 3 cartelle: **model**, contenente i file che si occupano di ottenere i dati utilizzati nell'applicazione, **view**, contenente i file che si occupano di aggiornare le pagine in base ai dati provenienti dal *model* e di fornire al *controller* alcune funzioni per gestire aspetti dell'interfaccia, e **controller**, contenente i file che si occupano di gestire le interazioni dell'utente e la navigazione tra le pagine dell'applicazione.

Le tre componenti interagiscono tra di loro nel seguente modo: all'avvio dell'applicazione, lo script *main* della pagina (*indexMain.js* o *hourlyWeatherMain.js*) invita il *model* a caricare i dati, quest'ultimo, dopo averli caricati, chiede al *view* di inserire i dati all'interno degli elementi della pagina.

Quando avviene un'interazione dell'utente essa viene processata dal *controller*, richiamando quando è necessario funzioni del *model* per cambiare i dati forniti (con il *model* che a sua volta richiamerà funzioni del *view* per ag-

giornare i dati visualizzati) o funzioni del *view* per apportare delle modifiche all'interfaccia. Alcune funzioni del *controller* utilizzano dati forniti dal *model*.

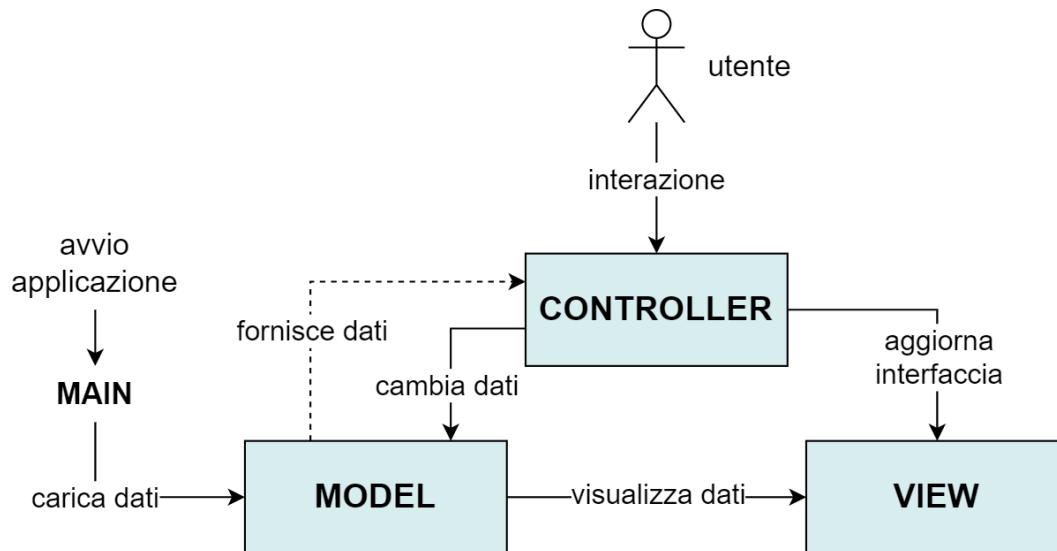


Figura 4.7: Interazione tra le componenti MVC

Al fine di migliorare l'isolamento e la gestione delle dipendenze tra i vari file, sono stati utilizzati i **moduli ES6**.

Nei prossimi paragrafi verranno illustrati i file contenuti all'interno delle tre componenti e quelli posizionati al di fuori di esse.

4.4.1 Model

dailyWeatherData.js

All'interno di questo file viene definito l'oggetto *dailyWeatherData*, contenente il metodo *fetchWeatherData()* che permette di caricare i dati riguardanti le previsioni giornaliere.

In questo metodo viene effettuata un'operazione *GET* (utilizzando la funzione *fetch()* di JavaScript) sulle API di *Open-Meteo*, specificando tra i para-

metri dell'URL i dati che si vogliono ottenere, le coordinate della città e le unità di misura.

Se la richiesta va a buon fine, la risposta ottenuta (che consiste in un JSON) viene memorizzata nel campo *data* dell'oggetto e viene passata come parametro al metodo *displayWeatherData()* dell'oggetto *dailyWeatherDataInserter* (importato dal modulo *dailyWeatherDataEntry.js*), che permette di visualizzare i dati.

hourlyWeatherData.js

In modo totalmente analogo a quanto descritto in *dailyWeatherData.js*, in questo file viene definito l'oggetto *hourlyWeatherData*, contenente il metodo *fetchWeatherData()* che permette di caricare i dati riguardanti le previsioni orarie.

Se la richiesta va a buon fine, i dati ottenuti vengono memorizzati nel campo *data* dell'oggetto e vengono passati come parametro al metodo *displayWeatherData()* dell'oggetto *hourlyWeatherDataInserter* (importato dal modulo *hourlyWeatherDataEntry.js*), che permette di visualizzare i dati.

geocodingData.js

Anche in questo caso viene definito un oggetto *geoData*, contenente il metodo *fetchGeocodingDataAndDisplaySuggestions()* che, utilizzando le API di geocodifica fornite sempre da *Open-Meteo*, permette di convertire l'input dell'utente, utilizzato per selezionare la città desiderata, in auto-completamenti suggeriti e di ottenere le relative coordinate geografiche.

Nell'URL utilizzato per effettuare l'operazione di *GET*, vengono inseriti come parametri l'input dell'utente (il testo inserito nella barra di ricerca), il numero massimo di risultati che si vogliono ottenere e la lingua.

In risposta si avrà un JSON che viene memorizzato nel campo *data* dell'oggetto e viene passato come parametro alla funzione *displaySuggestions()* (importata dal modulo *searchBarView.js*), che permette di visualizzare gli auto-completamenti.

4.4.2 View

dailyWeatherDataEntry.js

All'interno di questo file viene definito l'oggetto *dailyWeatherDataInserter*, che contiene tutti i metodi che si occupano di inserire i dati all'interno degli elementi della pagina *index.html*.

hourlyWeatherDataEntry.js

In questo file viene definito l'oggetto *hourlyWeatherDataInserter*, che contiene tutti i metodi necessari per inserire i dati all'interno della pagina *hourly-weather.html*.

searchBarView.js

Contiene la funzione *displaySuggestions()* utilizzata per mostrare gli auto-completamenti suggeriti all'utente. Inoltre è presente la funzione *highlightSelectedElement()* che permette di evidenziare l'autocompletamento selezionato.

viewTools.js

Fornisce dei metodi utilizzati in molteplici situazioni, ad esempio la funzione *getCurrentHourByTimezone()* che riceve in input una timezone e restituisce l'ora attuale in tale fascia.

Contiene inoltre il metodo *setMainContainerHeight()*, che fissa l'altezza del contenitore principale permettendo il comportamento *sticky* di *<aside>*.

4.4.3 Controller

`elementsEventListenersAndActions.js`

Questo file si occupa di impostare tutti gli *event listener* e le relative azioni da innescare riguardanti al click di alcuni elementi delle pagine, garantendo l'interattività dell'interfaccia.

Tra le funzioni principali presenti al suo interno abbiamo *setClickedMainInfoRowsActionIndexPath()*, che consente di accedere alla pagina *hourly-weather.html* cliccando la riga relativa alla giornata a cui si è interessati. I dati necessari per la corretta visualizzazione, come il nome e le coordinate geografiche della città, il giorno selezionato e la relativa ora dell'alba e del tramonto, vengono passati come parametro all'interno dell'URL.

Inoltre sono presenti le funzioni:

- *setClickedExpandImgActionIndexPath()* e *setClickedExpandImgActionHourlyWeatherPage()* che permettono di espandere/restringere il contenuto della tabella al click dell'apposita immagine nel layout mobile
- *setClickedMoreInfoImagesActions()* che permette di cambiare la data (in *index.html*) o l'ora (in *hourly-weather.html*) mostrata nella tabella a lato nel layout desktop al click dell'icona contenente il simbolo +
- *setLogosClickedAction()* che consente mediante il click dei loghi di ritornare alla pagina *index.html* quando ci si trova in *hourly-weather.html*, oppure semplicemente di ricaricare la pagina principale nel caso in cui ci si trova già in essa
- *setMinusButtonAction()* e *setPlusButtonAction()* che implementano il widget che garantisce la possibilità di cambiare il giorno di cui si vogliono visualizzare le previsioni orarie senza dover tornare alla pagina iniziale delle previsioni giornaliere

- *setClickedSettingsImgAction()*, *setClickedSaveSettingButtonAction()* e *setClickedFlagsImgAction()* che consentono di selezionare le impostazioni desiderate.

documentAndWindowEventListenersAndActions.js

Si occupa di impostare tutti gli *event listener* e le relative azioni da innescare riguardanti il documento e la finestra del browser. In particolare, per quanto riguarda il documento viene aggiunto un *event listener* al suo caricamento, che innesci alcune azioni necessarie all'avvio, mentre alla finestra viene aggiunto un *event listener* al suo resize, che innesci delle azioni utilizzate a supporto della responsivness dell'interfaccia.

searchBarEventListenersAndActions.js

Questo file contiene delle funzioni che aggiungono gli *event listener* ed impostano le relative azioni da innescare necessari per:

- aggiornare gli auto-completamenti suggeriti
- consentire all'utente di spostarsi tra i suggerimenti tramite i tasti freccia della tastiera
- permettere all'utente di selezionare l'auto-completamento desiderato.

controllerTools.js

Contiene la funzione *isIndexPage()*, che restituisce *true* se ci si trova nella pagina *index.html*.

4.4.4 File esterni alle componenti

indexMain.js

Richiama le funzioni necessarie quando avviene l'accesso alla pagina *index.html*.

hourlyWeatherMain.js

Richiama le funzioni necessarie quando avviene l'accesso alla pagina *hourly-weather.html*.

settings.js

Contiene delle funzioni utilizzate per fissare alcune impostazioni.

parametersVariablesHourlyWeather.js

Al suo interno si trova la funzione che permette di estrapolare dall'URL i parametri passati e memorizzarli in apposite variabili.

constantsAndMultipurposeVariables.js

Contiene le costanti e le variabili utilizzate in molteplici file, con le relative funzioni per modificare quest'ultime.

Capitolo 5

Risultato finale ed analisi di alcuni casi d'uso

In questo capitolo sono riportate alcune schermate del risultato finale ottenuto a seguito dell'implementazione.

Quando l'utente accede all'applicazione visualizzerà la pagina *index.html*, contenente le previsioni giornaliere riguardanti l'ultima città ricercata, utilizzando le ultime impostazioni fissate. Nel caso in cui queste informazioni non siano salvate (ad esempio se l'utente non ha mai utilizzato l'applicazione prima) verranno visualizzate in automatico le previsioni relative alla città di Perugia, in lingua italiana ed utilizzando unità di misura predefinite.

Nel layout desktop, nella tabella a lato vengono visualizzati di default i dati secondari relativi alla giornata odierna. L'utente può cambiare il giorno di cui vuole visualizzare i dati cliccando l'icona contenente il simbolo + posizionata alla fine della relativa riga. Come si può vedere nella figura 5.8, la tabella rimane visibile anche quando si scorre la pagina.

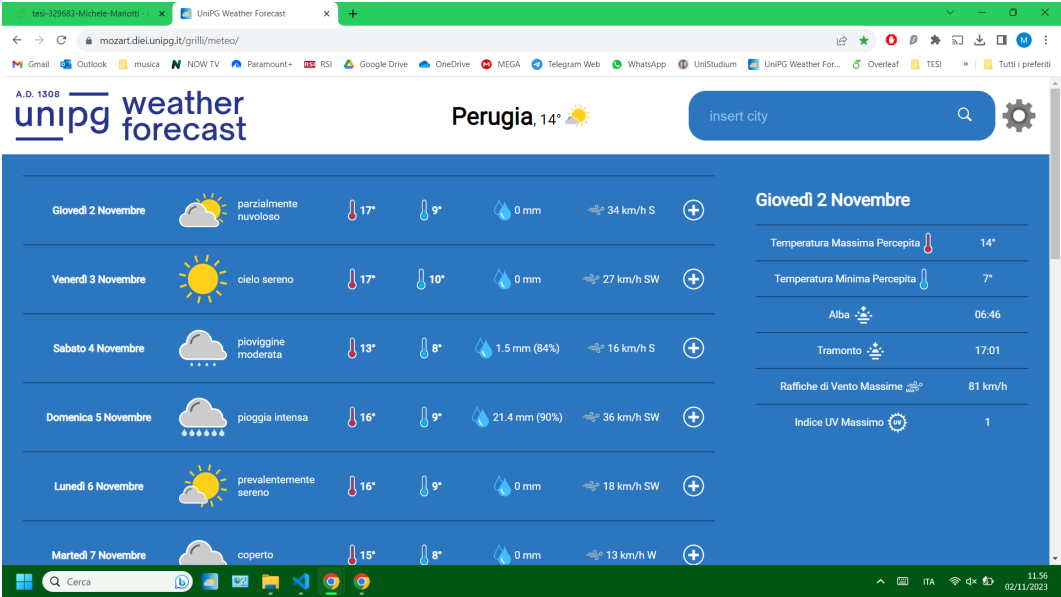


Figura 5.1: Pagina iniziale previsioni giornaliere layout desktop

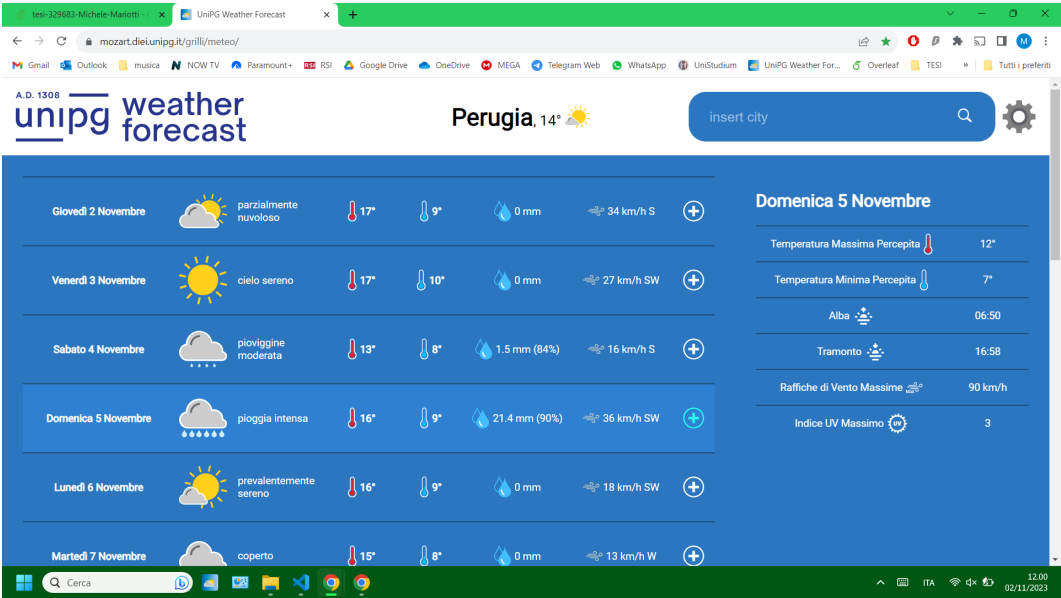
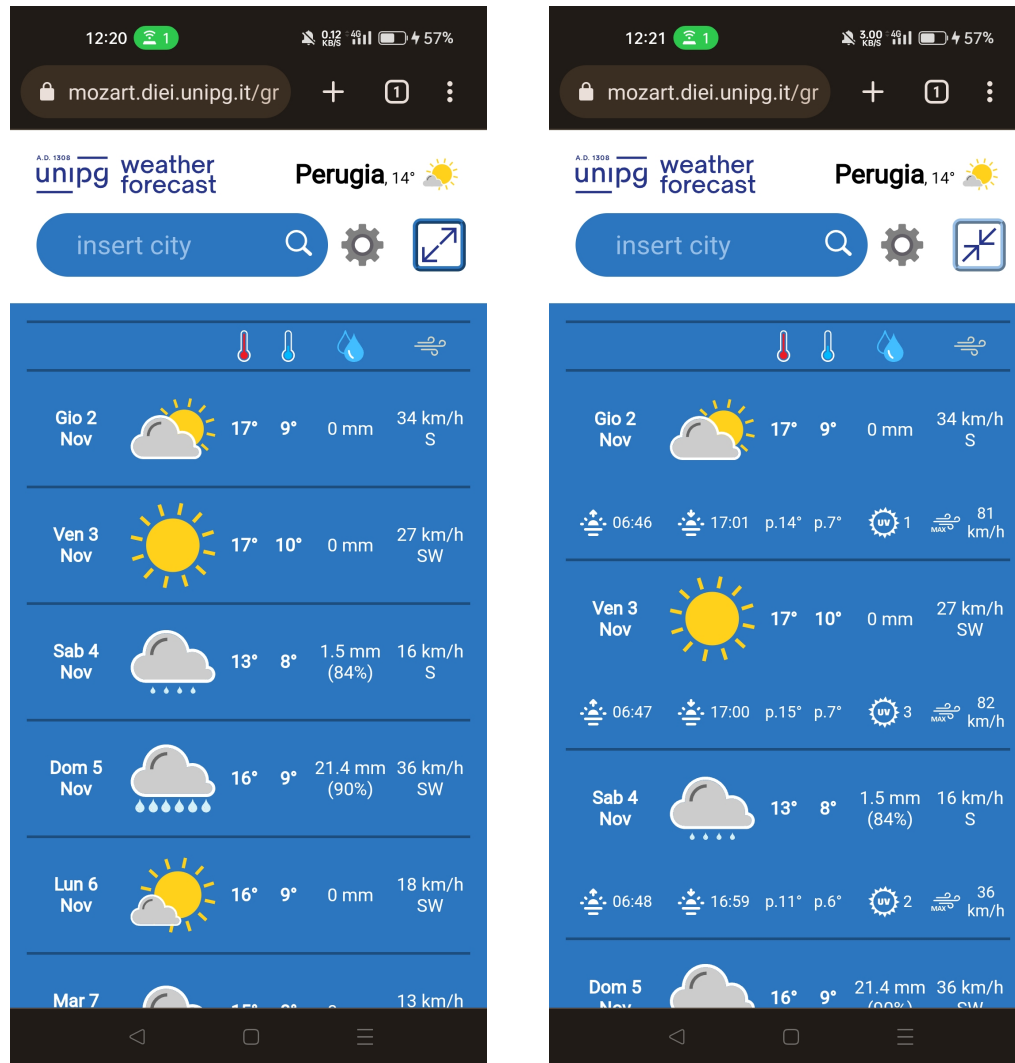


Figura 5.2: Selezione giornata dati secondari

Nel caso del layout mobile di default i dati secondari vengono nascosti, per visualizzarli è sufficiente cliccare l'icona “espandi”. Successivamente per tornare alla visuale iniziale ridotta è sufficiente cliccare l'icona “restringi”.



(a) Versione “ristretta”

(b) Versione “espansa”

Figura 5.3: Pagina iniziale previsioni giornaliere layout mobile

In entrambi i layout è possibile visualizzare le impostazioni cliccando l'icona dell'ingranaggio. Una volta selezionate le impostazioni desiderate, cliccando

il pulsante “save” l'applicazione ricaricherà la pagina iniziale utilizzando le impostazioni fissate.

Per cambiare la città di cui si vogliono visualizzare le previsioni meteo si deve digitare nell'apposita barra di ricerca il nome della città desiderata. Ad ogni carattere digitato vengono mostrati i possibili auto-completamenti tra cui scegliere (selezionabili anche tramite tasti freccia e tasto enter).



(a) Impostazioni

(b) Auto-completamento ricerca

Figura 5.4: Impostazioni ed auto-completamento ricerca su dispositivi mobili

Cliccando all'interno della riga relativa alla giornata a cui si è interessati si accede alla pagina delle previsioni orarie.

Nel layout desktop, nella tabella a lato vengono visualizzati di default i dati secondari relativi alla prima ora disponibile (00:00 nel caso in cui la data selezionata non sia quella odierna). L'utente può cambiare l'orario di cui vuole visualizzare i dati cliccando l'icona contenente il simbolo + posizionata alla fine della relativa riga.

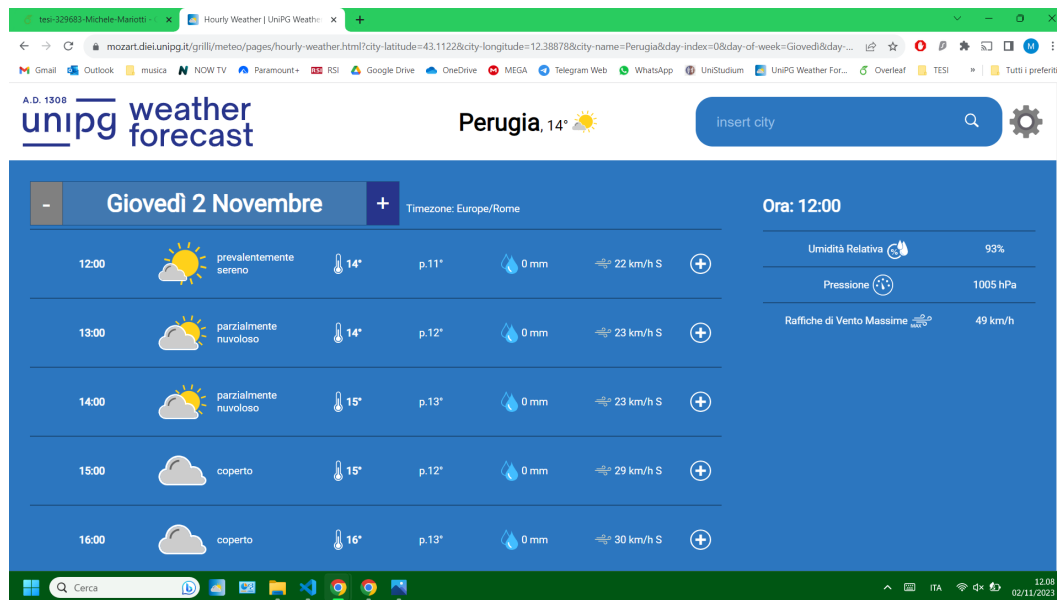


Figura 5.5: Pagina previsioni orarie layout desktop

Per quanto riguarda il layout mobile, analogamente a quanto visto per la pagina principale, per visualizzare i dati secondari è necessario cliccare l'icona "espandi".

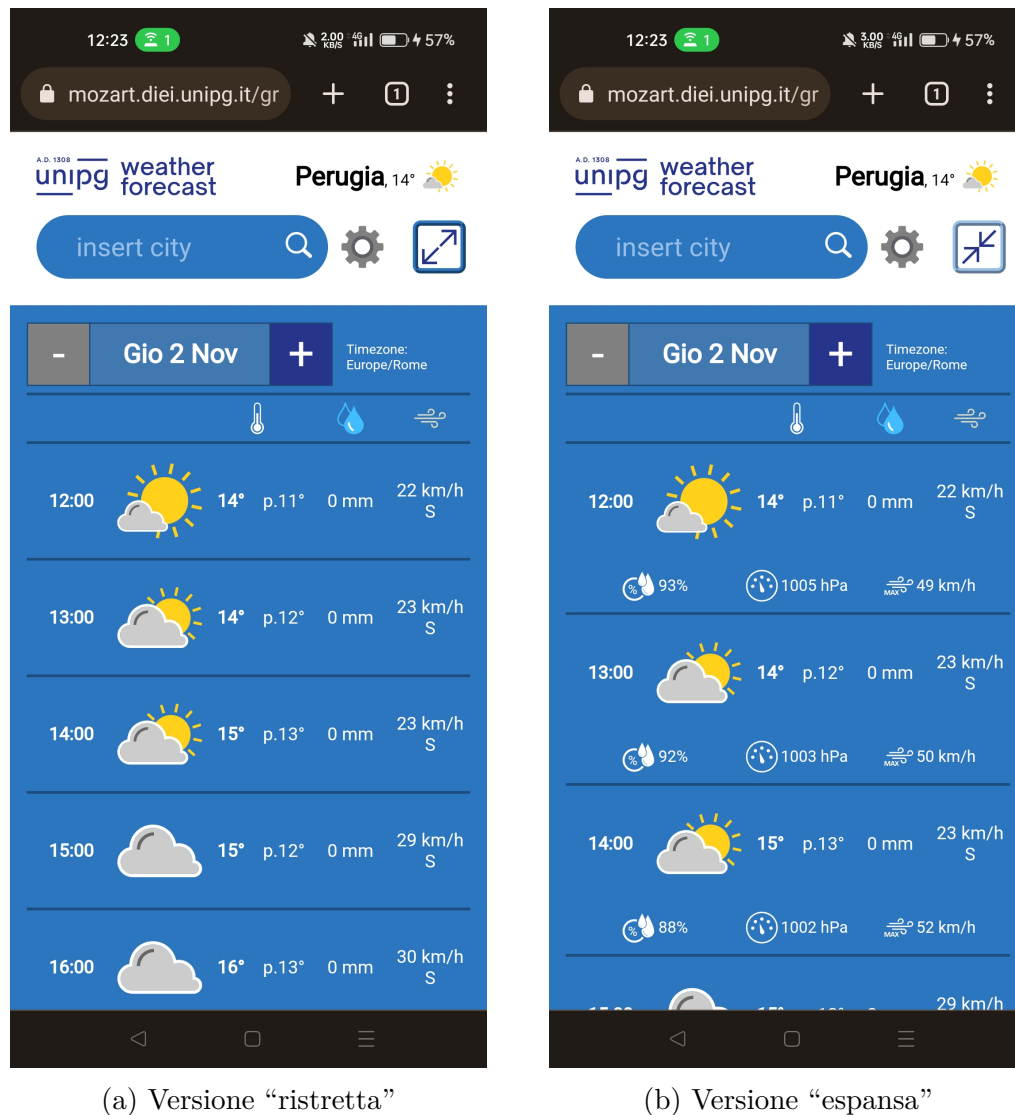


Figura 5.6: Pagina previsioni orarie layout mobile

Attraverso il widget è possibile cambiare la giornata selezionata con la precedente o la successiva. Nella prima giornata (data odierna) il pulsante per passare alla data precedente è disabilitato mentre nell'ultima giornata disponibile è il tasto che permette di accedere alla data successiva ad essere disabilitato.

Le icone delle condizioni meteo tengono in considerazione gli orari dell'alba

e del tramonto, mostrando le icone con la luna, invece che con il sole, prima dell'alba e dopo il tramonto. Allo stesso modo anche l'icona della condizione meteo attuale posizionata nell'header varia in base all'orario attuale.



Figura 5.7: Icone condizioni meteo prima e dopo l'alba

In fondo alle pagine è presente un footer, contenente informazioni generali sull'applicazione ed il logo dell'ateneo. Cliccando sul logo, si viene reindirizzati al relativo portale d'ateneo.

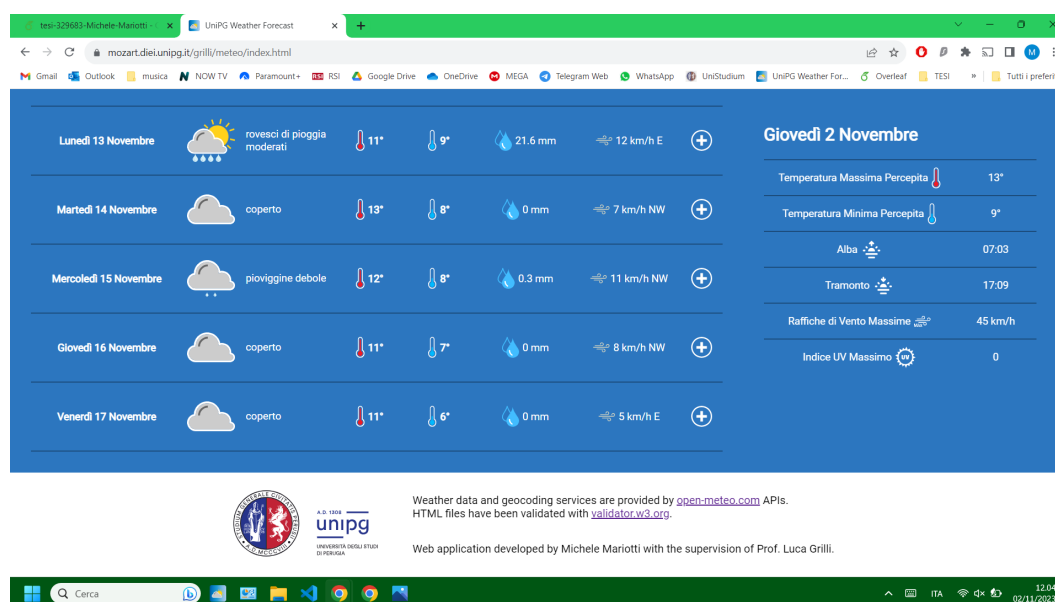


Figura 5.8: Footer della pagina

Capitolo 6

Test

6.1 Validazione HTML

I file HTML dell'applicazione sono stati validati attraverso la piattaforma *validator.w3.org* [16] ottenendo il “bollino verde”. Questo strumento è utile per garantire che l'applicazione Web sia ben strutturata e possa essere visualizzata correttamente sui diversi browser e dispositivi.

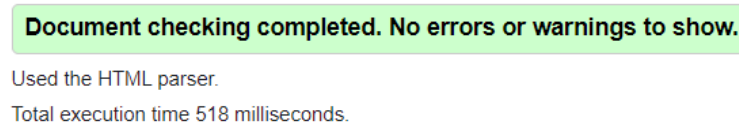


Figura 6.1: “Bollino verde” validazione HTML

6.2 Test di efficienza

Al fine di analizzare e valutare le prestazioni dell'applicazione è stata utilizzata la piattaforma online *GTmetrix* [17]. Per avviare il test è sufficiente inserire l'URL della pagina da analizzare e scegliere le impostazioni desiderate. Alla

fine dell'esecuzione del test viene fornito un report contenente vari dati, tra cui:

- **Punteggio GTmetrix:** valutazione complessiva della pagina. È ottenuto attraverso la media ponderata di due punteggi espressi in percentuale: il punteggio delle prestazioni (con un peso del 60%) ed il punteggio della struttura (con un peso del 40%). Il punteggio percentuale ottenuto viene convertito in una valutazione espressa con una lettera tra A e F in base a dei range prestabiliti.
- **Risultati Web Vitals:** metriche di prestazione Web introdotte da Google. Queste metriche si concentrano sugli aspetti chiave per valutare l'esperienza fornita dalla pagina, in particolare:
 - **LCP (Largest Contentful Paint):** misura il tempo necessario affinché l'elemento di contenuto più grande della pagina diventi visibile nel viewport dei visitatori.
 - **TBT (Total Blocking Time):** indica per quanto tempo il processo di caricamento della pagina è bloccato dagli script.
 - **CLS (Cumulative Layout Shift):** misura lo spostamento del layout durante il caricamento della pagina.
- **Dettagli della pagina:** mostra informazioni sul tipo e la dimensione delle richieste fatte dalla pagina. Viene riportato il tempo di caricamento totale della pagina.
- **Principali problemi:** vengono riportati i principali problemi riscontrati ed ipotizzate possibili soluzioni.

Il test è stato effettuato sulla pagina principale delle previsioni giornaliere, utilizzando il server localizzato a Londra (la posizione disponibile più vicina

all'Italia), simulando una connessione a banda larga con una velocità di 5 Mbps in download, 1 Mbps in upload ed una latenza di 30 ms.

I risultati ottenuti sono stati soddisfacenti. Nella figura 6.2 vengono riportati i principali dati del test in una tabella comparativa con i dati dei test svolti sulle pagine con le previsioni giornaliere delle principali piattaforme che offrono lo stesso servizio.

	UniPG Weather Forecast https://mozart.diei.unipg.it/grilli/... Fri, Oct 27, 2023 1:54 AM -0700 London, UK Chrome (Desktop) 117.0.0.0 Broadband (5/1 Mbps, 30ms)	meteoam https://www.meteoam.it/it/met... Fri, Oct 27, 2023 1:33 AM -0700 London, UK Chrome (Desktop) 117.0.0.0 Broadband (5/1 Mbps, 30ms)	meteo.it https://www.meteo.it/meteo/p... Fri, Oct 27, 2023 1:23 AM -0700 London, UK Chrome (Desktop) 117.0.0.0 Broadband (5/1 Mbps, 30ms)
GTmetrix Grade	A (96%)	F (40%)	D (63%)
Performance Score	99%	20% -79%	60% -39%
Structure Score	92%	71% -21%	67% -25%
Largest Contentful Paint	841ms	6.0s +5.2s	1.8s +982ms
Total Blocking Time	0ms	2.0s +2.0s	299ms +299ms
Cumulative Layout Shift	0	0.18 +0.18	0.23 +0.23
Total Page Size	166KB	2.33MB +2.17MB	10.8MB +10.7MB
Total # of Requests	51	213 +162	487 +436

	UniPG Weather Forecast https://mozart.diei.unipg.it/grilli/... Fri, Oct 27, 2023 1:54 AM -0700 London, UK Chrome (Desktop) 117.0.0.0 Broadband (5/1 Mbps, 30ms)	Il Meteo https://www.ilmeteo.it/meteo/P... Fri, Oct 27, 2023 1:41 AM -0700 London, UK Chrome (Desktop) 117.0.0.0 Broadband (5/1 Mbps, 30ms)	3Bmeteo https://www.3bmeteo.com/met... Fri, Oct 27, 2023 1:36 AM -0700 London, UK Chrome (Desktop) 117.0.0.0 Broadband (5/1 Mbps, 30ms)
GTmetrix Grade	A (96%)	B (82%)	D (68%)
Performance Score	99%	88% -11%	54% -45%
Structure Score	92%	74% -18%	90% -2%
Largest Contentful Paint	841ms	796ms -44ms	1.2s +399ms
Total Blocking Time	0ms	190ms +190ms	394ms +394ms
Cumulative Layout Shift	0	0.03 +0.03	0.4 +0.4
Total Page Size	166KB	2.67MB +2.51MB	1.58MB +1.42MB
Total # of Requests	51	126 +75	77 +26

Figura 6.2: Tabelle comparative risultati test

Capitolo 7

Conclusioni e sviluppi futuri

Questo progetto di tesi è stato dedicato allo sviluppo di un'applicazione Web responsive per la visualizzazione di previsioni meteorologiche.

Inizialmente è stato esplorato l'interesse degli utenti nel consultare le informazioni meteorologiche in modo chiaro ed immediato. La loro richiesta è quella di avere un'interfaccia esente da pubblicità invasiva che fornisca un quadro sintetico delle previsioni, consentendo loro di accedere alle informazioni desiderate in modo efficiente. Questo desiderio è stato posto come obiettivo del progetto di tesi.

Per realizzare il progetto è stato necessario apprendere e prendere dimestichezza con alcuni concetti e tecnologie di estrema importanza per lo sviluppo Web.

Successivamente è stata condotta un'analisi della problematica affrontata, la quale ha guidato la selezione della piattaforma utilizzata per reperire le previsioni meteorologiche e la determinazione dei dati forniti. Inoltre sono state tracciate le linee guida definite nella specifica dei requisiti.

A seguire sono stati realizzati i wireframe, in cui è stata definita la modalità con cui i dati vengono mostrati.

A questo punto l'idea progettata è stata concretizzata attraverso l'utilizzo di un'architettura ben definita. La struttura delle pagine è stata implementata utilizzando il linguaggio HTML, per poi passare alla definizione dello stile attraverso il linguaggio CSS ed infine alla realizzazione del comportamento e dell'interattività dell'applicazione tramite il linguaggio di programmazione JavaScript.

Sono stati poi fatti dei test per valutare alcuni aspetti dell'applicazione che hanno dato risultati soddisfacenti. L'applicazione realizzata è fedele all'idea iniziale e rispecchia i wireframe elaborati preliminarmente. Sono stati rispettati i requisiti e gli obiettivi prefissati, realizzando un'applicazione Web con un'interfaccia responsive, user-friendly e priva di pubblicità.

7.1 Sviluppi futuri

Nonostante ciò, ci sono sempre possibilità di miglioramento. Ad esempio è possibile ipotizzare i seguenti sviluppi futuri:

- inclusione di ulteriori dati, come lo storico del meteo passato, utile per valutare il cambiamento climatico, dati sulla qualità dell'aria, utili per chi soffre di allergie e per valutare l'inquinamento, e previsioni del meteo marine riportanti dati sulle onde.
- possibilità di personalizzazione, realizzando dei temi grafici tra cui l'utente può scegliere, o rendendo possibile una completa customizzazione dei colori dell'interfaccia.
- miglioramento del disaccoppiamento della componente *model* dal *view* e dal *controller*, rendendo possibile un futuro cambiamento della piattaforma utilizzata per reperire i dati senza dover fare nessun cambiamento

nel codice del *view* e del *controller*, indipendentemente dalla struttura del JSON restituito.

- aggiunta della possibilità di scegliere il fuso orario, che attualmente corrisponde a quello della città ricercata.

Bibliografia

- [1] *Recensioni di uno dei principali portali per le previsioni meteorologiche*, <https://it.trustpilot.com/review/meteoam.it>
- [2] *Responsive design*, Mozilla, https://developer.mozilla.org/en-US/docs/Learn/CSS/CSS_layout/Responsive_Design
- [3] *Immagine esempio interfaccia responsive*, <https://www.firstclasswebdesign.co.uk/mobile-friendly-website-design/>
- [4] *HTML*, Mozilla, <https://developer.mozilla.org/en-US/docs/Web/HTML>
- [5] *CSS*, Mozilla, <https://developer.mozilla.org/en-US/docs/Web/CSS>
- [6] *Immagine media queries*, <https://kinsta.com/it/wp-content/uploads/sites/2/2020/09/interrogazioni-multimediali.png>
- [7] *JavaScript*, Mozilla, <https://developer.mozilla.org/en-US/docs/Web/JavaScript>
- [8] *JavaScript modules*, Mozilla, <https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Modules>
- [9] *Moduli ECMAScript 2015*, *html.it*, <https://www.html.it/pag/64435/moduli-2/>

-
- [10] *API REST, IBM*, <https://www.ibm.com/it-it/topics/rest-apis>
 - [11] *JSON, W3Schools*, https://www.w3schools.com/js/js_json.asp
 - [12] *Data Source*, <https://open-meteo.com/en/docs>
 - [13] *Tabella fornitori nazionali dati meteo*, <https://open-meteo.com/en/docs>
 - [14] *Immagine conversione gradi in punti cardinali*, <https://www.surfertoday.com/windsurfing/how-to-read-wind-direction>
 - [15] *Mobile First, Mozilla*, https://developer.mozilla.org/en-US/docs/Glossary/Mobile_First
 - [16] *Piattaforma per validazione HTML: validator.w3.org*, <https://validator.w3.org/>
 - [17] *Piattaforma per analizzare performance applicazioni e siti: gtmetrix.com*, <https://gtmetrix.com/>